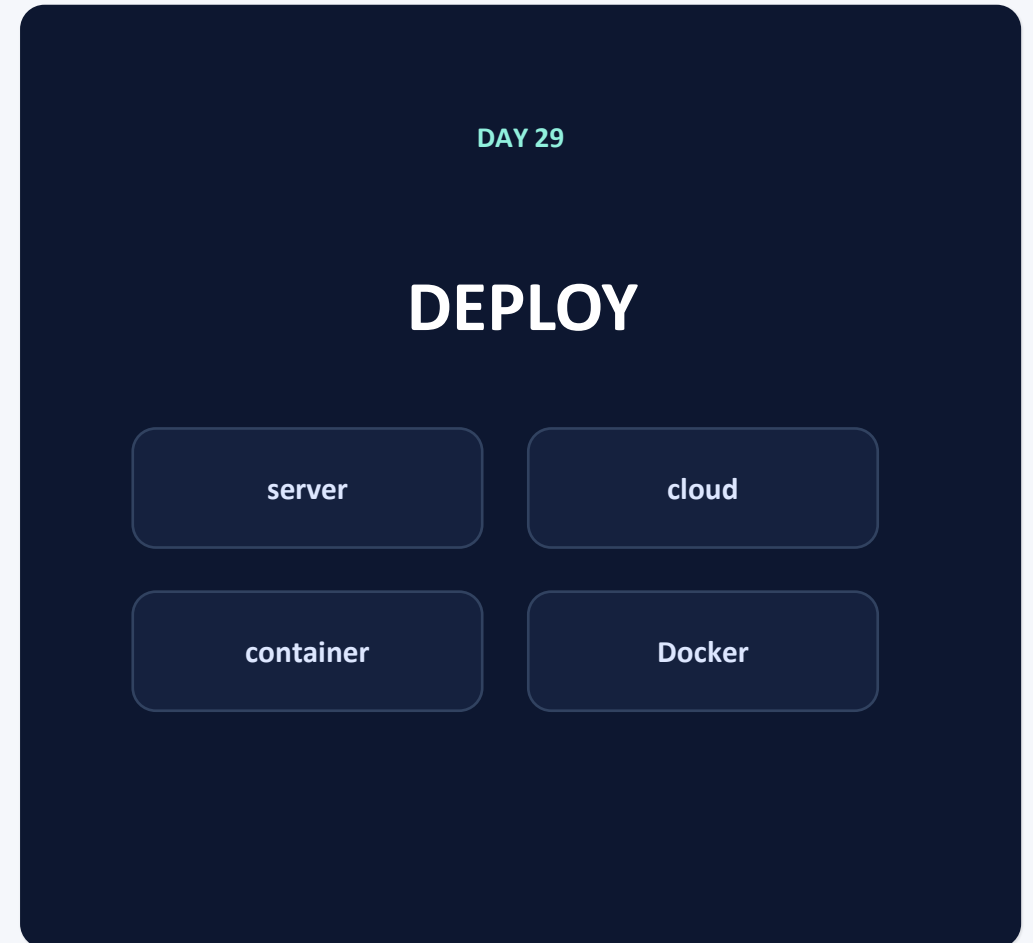


Деплой, Docker, облако и CI/CD

Понять путь от кода до работающего сервиса: Docker image/container, environment, cloud/server, CI/CD pipeline, health check и rollback.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Понять путь от кода до работающего сервиса: Docker image/container, environment, cloud/server, CI/CD pipeline, health check и rollback.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Безопасность приложений

Что уже есть в нашей модели.



СЕГОДНЯ

Деплой, Docker, облако и CI/CD

Вчера: Безопасность приложений. Сегодня мы добавляем следующий слой: деплой, docker, облако и ci/cd.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Пользователь получает ценность только после доставки кода в рабочую среду. Повторяемый deploy снижает количество «у меня работает» проблем.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Docker image — запечатанная коробка с приложением, **container** — запущенный экземпляр коробки, **CI/CD** — конвейер проверки и доставки.

ПЕРЕВОД В IT

deploy

доставка версии приложения в целевую среду

server

система, предоставляющая сервис другим программам

cloud

удалённые вычислительные ресурсы как услуга

Слой 1: deploy • server • cloud

Сначала смысл, затем профессиональный термин.

**deploy**

доставка версии приложения в целевую среду

**server**

система, предоставляющая сервис другим программам

**cloud**

удалённые вычислительные ресурсы как услуга

Слой 2: container • Docker • CI/CD

Сначала смысл, затем профессиональный термин.

**container**

изолированный пакет приложения и его окружения

**Docker**

платформа сборки и запуска контейнеров

**CI/CD**

автоматизация проверок, подготовки и доставки изменений

Слой 3: image • pipeline

Сначала смысл, затем профессиональный термин.

image

шаблон файлов и настроек для создания контейнера

pipeline

автоматическая последовательность сборки и проверки

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: environment • rollback

Сначала смысл, затем профессиональный термин.

environment

окружение с настройками и ресурсами приложения

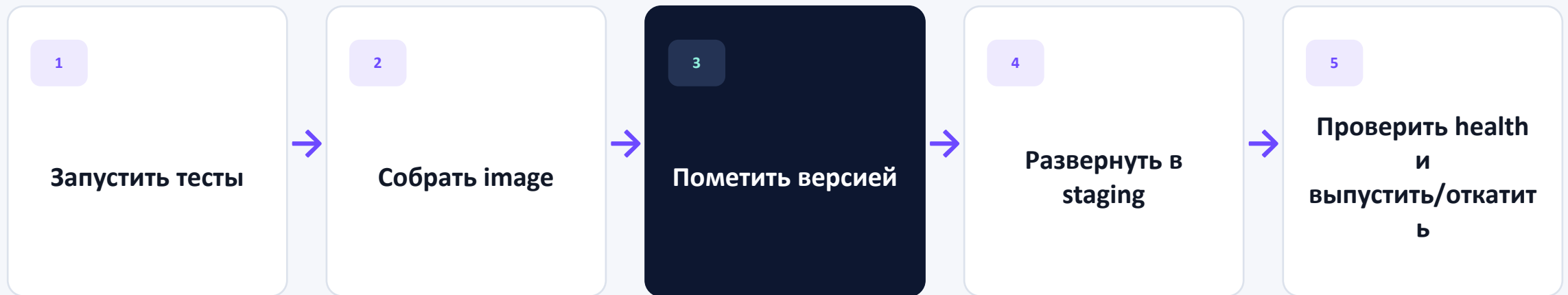
rollback

возврат развёртывания к рабочей версии

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

day29_step1

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

FROM python:3.12-slim

Python выполняет эту строку сверху вниз.

WORKDIR /app

Python выполняет эту строку сверху вниз.

COPY requirements.txt .

Python выполняет эту строку сверху вниз.

RUN pip install -r requirements.txt

Python выполняет эту строку сверху вниз.

COPY . .

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

РЕЗУЛЬТАТ

Docker image содержит Python, зависимости и приложение

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
docker build -t myapp:1.0 .  
docker run --rm -p 8000:8000 myapp:1.0
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Контейнер запускается из конкретной версии image

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
FROM python:latest  
COPY . .  
RUN pip install -r requirements.txt
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

latest делает базовую версию плавающей, а ранний COPY всего проекта ухудшает кеширование и может тащить лишние файлы. Лучше фиксировать версию и сначала копировать зависимости.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Спроектировать безопасный pipeline: тесты → build → staging → health check → production.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **CI запускает pytest.**
- 2 При успехе собирается versioned image.
- 3 Image отправляется в registry.
- 4 Разворачивается staging и проходит health check.
- 5 Только затем production; при проблеме — rollback на прошлую версию.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

1. pytest
2. docker build -t app:1.4.0 .
3. push app:1.4.0
4. deploy staging

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

- 
- 
1. pytest
 2. docker build -t app:1.4.0 .
 3. push app:1.4.0
 4. deploy staging
 5. health check
 6. deploy production
 7. rollback -> app:1.3.2 if needed

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Каждый релиз воспроизводим и имеет понятную предыдущую версию

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Почему тег ``latest`` хуже версии ``1.4.0`` при расследовании проблем?

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Нужна однозначная связь между запущенным кодом и конкретным артефактом.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



```
docker image tag myapp:latest myapp:1.4.0  
# в production используем myapp:1.4.0
```

practice_solution

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Можно точно понять, какая версия развернута

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Собери план rollback после неуспешного health check новой версии.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
deploy app:1.4.0  
health check -> FAIL  
stop rollout  
deploy app:1.3.2  
health check -> OK  
investigate 1.4.0
```

The image shows a terminal window with a dark blue background. In the top-left corner, there are three colored circles (red, yellow, green) representing a window's title bar. In the top-right corner, the text 'mini_project' is displayed. The main area of the terminal contains a sequence of commands and their outputs: 'deploy app:1.4.0', 'health check -> FAIL', 'stop rollout', 'deploy app:1.3.2', 'health check -> OK', and 'investigate 1.4.0'.

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Сервис возвращён на последнюю известную рабочую версию

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Разворачивать непроверенный image прямо в production.

02

Использовать плавающие версии без необходимости.

03

Хранить секреты внутри image.

04

Не иметь наблюдаемости и понятного rollback.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «deploy» и какую проблему это понятие помогает описать?

2

Чем «deploy» отличается от «container»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **deploy — доставка версии приложения в целевую среду.**

2 **deploy: доставка версии приложения в целевую среду. container: изолированный пакет приложения и его окружения.**

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Docker image содержит Python, зависимости и приложение

4 latest делает базовую версию плавающей, а ранний COPY всего проекта ухудшает кеширование и может тащить лишние файлы. Лучше фиксировать версию и сначала копировать зависимости.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Понять путь от кода до работающего сервиса: Docker image/container, environment, cloud/server, CI/CD pipeline, health check и rollback.
- ✓ Читать и объяснять пример темы «Деплой, Docker, облако и CI/CD».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Профессия, портфолио и следующий шаг

Завтра: Профессия, портфолио и следующий шаг. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.